

PHASE_0_ARCHITECTURE

- [SUOP ERP v1.0 — Phase 0: Enterprise Foundation Architecture](#)
 - [Table of Contents](#)
 - [1. Purpose and Scope](#)
 - [2. Architectural Principles](#)
 - [3. Folder Structure](#)
 - [4. Layered Architecture](#)
 - [5. Dependency Graph](#)
 - [6. Shared Libraries](#)
 - [7. Base Classes and Interfaces](#)
 - [8. Middleware Stack](#)
 - [9. Workflow Engine](#)
 - [10. Notification Engine](#)
 - [11. Audit Framework](#)
 - [12. Validation Framework](#)
 - [13. Error Framework](#)
 - [14. Configuration Framework](#)
 - [15. Transaction Helper](#)
 - [16. File Upload Service](#)
 - [17. Background Job Framework](#)
 - [18. Event Bus](#)
 - [19. Shared Types and Enums](#)
 - [20. Shared DTOs](#)
 - [21. Shared Domain Events](#)
 - [22. Testing Strategy](#)
 - [23. Coding Standards](#)
 - [24. Definition of Done — Phase 0](#)
 - [25. Open Questions for Approval](#)
 - [Approval Block](#)

SUOP ERP v1.0 — Phase 0:

Enterprise Foundation

Architecture

Field	Value
Document Version	1.0
Status	DRAFT — Awaiting Stakeholder Approval
Author	Lead Software Architect, SUOP v1.0
Date	2026-07-11
Supersedes	All prior implementation decisions
Approval Required From	Project Owner

Table of Contents

- 1. [Purpose and Scope](#)
- 2. [Architectural Principles](#)
- 3. [Folder Structure](#)
- 4. [Layered Architecture](#)
- 5. [Dependency Graph](#)
- 6. [Shared Libraries](#)
- 7. [Base Classes and Interfaces](#)
- 8. [Middleware Stack](#)
- 9. [Workflow Engine](#)
- 10. [Notification Engine](#)
- 11. [Audit Framework](#)
- 12. [Validation Framework](#)
- 13. [Error Framework](#)
- 14. [Configuration Framework](#)

15. [Transaction Helper](#)
 16. [File Upload Service](#)
 17. [Background Job Framework](#)
 18. [Event Bus](#)
 19. [Shared Types and Enums](#)
 20. [Shared DTOs](#)
 21. [Shared Domain Events](#)
 22. [Testing Strategy](#)
 23. [Coding Standards](#)
 24. [Definition of Done — Phase 0](#)
 25. [Open Questions for Approval](#)
-

1. Purpose and Scope

1.1 Purpose

This document defines the **enterprise foundation architecture** for SUOP ERP v1.0. It establishes the shared platform upon which every future ERP module — Organization, Auth, Procurement, Inventory, Manufacturing, Quality, Finance, Retail, Restaurant, and AI — will be built.

The foundation is built **once** and **reused** by every module. No module implementation may bypass or duplicate the foundation. If a module appears to need functionality the foundation does not provide, the correct response is to **extend the foundation**, not to build a parallel mechanism inside the module.

1.2 Scope

Phase 0 covers the following platform capabilities. **No business module code is written in Phase 0.**

- Folder structure and project layout
- Layered architecture and dependency rules
- Shared libraries and base classes
- Middleware stack (auth, RBAC, tenant, audit, validation, error, logging, rate-limit)

- Workflow engine (state machine framework)
- Notification engine (in-app, email, SMS, WhatsApp)
- Audit framework (immutable audit trail)
- Validation framework (schema + business rule)
- Error framework (typed errors, error codes, response envelope)
- Configuration framework (env validation, feature flags)
- Transaction helper (Prisma transaction wrapper)
- File upload service (S3-compatible)
- Background job framework (cron + queued jobs)
- Event bus (in-process pub/sub for domain events)
- Shared types, enums, DTOs, events
- Testing strategy and infrastructure
- Coding standards

1.3 Non-Goals

Phase 0 does **not** include:

- Database schema changes (separate Schema Review Report will be produced)
- Stock Ledger architecture (separate dedicated document will be produced)
- Any business module implementation
- Migration of existing mock code
- UI changes

1.4 Approval Gate

Phase 0 deliverables are **documentation only**. No production code is written until this document is approved in full. After approval, implementation of Phase 0 foundation code begins, and only after Phase 0 code is complete and tested may Phase 1 (Organization module) begin.

2. Architectural Principles

2.1 Layered Architecture with Strict Dependency Direction

```
HTTP Request
  ↓
Routes (thin – only URL → controller mapping)
  ↓
Controllers (thin – request parsing, response shaping)
  ↓
Services (business logic, orchestration, workflows)
  ↓
Repositories (data access, Prisma queries)
  ↓
Database (PostgreSQL via Prisma)
```

Rule: Dependencies flow downward only. A repository may never import a service. A service may never import a controller. A controller may never call Prisma directly.

Enforcement: ESLint `no-restricted-imports` rule enforces layer boundaries at lint time. CI fails on violation.

2.2 Reuse Over Duplication

Any logic that will be needed by more than one module must be promoted to the foundation. This includes:

- CRUD operations → `BaseRepository`
- Validation → `ValidationFramework` + zod schemas
- State transitions → `WorkflowEngine`
- Audit logging → `AuditFramework` (automatic via middleware)
- Notifications → `NotificationEngine` (fired via domain events)
- Pagination, filtering, sorting → `BaseService.list()` + shared DTOs
- Permission checks → `@requirePermission` decorator + RBAC middleware

2.3 State Machine for Every Lifecycle

No status field may be mutated directly. Every entity with a lifecycle (PurchaseOrder, GRN, NCR, CAPA, COA, ProductionOrder, Batch, etc.) must declare a state machine using the `WorkflowEngine`. The engine validates transitions, fires pre/post hooks, emits domain events, and writes audit logs.

2.4 Multi-Tenancy from Day One

Every business table includes `tenantId`. Every repository query automatically scopes by tenant. Cross-tenant access requires explicit elevation and is audit-logged at a higher severity.

2.5 Optimistic Concurrency

Every business table includes `version Int @default(0)`. Updates must use Prisma's `updateMany` with `where: { id, version }` and increment version. Conflicts return `409 Conflict` with the current version.

2.6 Soft Delete with Immutable Audit

No business record is hard-deleted. Soft delete sets `deletedAt` and `deletedBy`. Soft-deleted records are filtered out by default in repositories but remain queryable for audit. True deletion requires a separate “purge” operation that itself writes an audit record describing what was purged and why.

2.7 Event-Driven Side Effects

Side effects (notifications, cache invalidation, downstream updates) are triggered by domain events published on the Event Bus — not by direct service-to-service calls. This decouples modules and makes the system observable.

2.8 Fail Fast, Fail Loud

Missing configuration, missing dependencies, schema drift, and contract violations must cause the application to refuse to start, not silently fall back to demo mode. Every `process.env` read goes through the Configuration Framework, which validates at boot.

2.9 Testability is Non-Negotiable

Every foundation component ships with unit tests. Every public method on a base class or framework has at least one test per branch. Coverage gate in CI: foundation code $\geq 90\%$, business code $\geq 70\%$.

2.10 No Magic, No Surprises

Code must be readable without IDE tooling. Avoid exotic TypeScript patterns, runtime type manipulation, or implicit behavior. Decorators are allowed where they reduce boilerplate (e.g., `@requirePermission`) but must be documented.

3. Folder Structure

3.1 Monorepo Layout

```
suop-erp/
├─ apps/
│   ├─ web/                # Next.js 16 frontend
│   └─ backend/            # Bun + Hono backend
service
│   └─ mobile/             # React Native + Expo
│
├─ packages/
│   └─ shared/              # Shared types, enums,
DTOs, events
│   └─ prisma/              # Prisma schema,
migrations, client
```

```

|   └─ ui/                                # Shared UI component
library
|   └─ eslint-config/                    # Shared ESLint config
|
└─ docs/                                # Architecture, ADRs,
runbooks
|   └─ architecture/
|       └─ PHASE_0_ARCHITECTURE.md
|       └─ SCHEMA_REVIEW_REPORT.md
|       └─ STOCK_LEDGER_DESIGN.md
|   └─ adr/                              # Architecture
Decision Records
|   └─ runbooks/
|
└─ docker/
    └─ Dockerfile.web
    └─ Dockerfile.backend
    └─ docker-compose.yml
|
└─ .github/
    └─ workflows/
        └─ ci.yml
        └─ cd-staging.yml
        └─ cd-production.yml
|
└─ package.json                          # Workspace root
└─ turbo.json                            # Turborepo config
└─ README.md

```

Rationale: Monorepo with Turborepo. `apps/` contains deployable applications. `packages/` contains shared code consumed by multiple apps. `packages/shared` is the single source of truth for types and contracts between frontend and backend.

3.2 Backend Folder Structure (apps/backend/)

```
apps/backend/
├─ src/
│   └─ main.ts                    # Bootstrap: load
│   config, connect DB, start server
│   └─ app.ts                    # Hono app
│   composition, middleware registration
│   |
│   └─ config/                  # Configuration
framework
│   |
│   └─ env.ts                   # Env validation (zod)
│   |
│   └─ features.ts              # Feature flags
│   |
│   └─ index.ts
│   |
│   └─ core/                    # Foundation framework
│   |   └─ errors/              # Error framework
│   |       └─ base-error.ts
│   |       └─ error-codes.ts
│   |       └─ error-handler.ts
│   |       └─ index.ts
│   |
│   |   └─ response/            # Response envelope
│   |       └─ envelope.ts
│   |       └─ index.ts
│   |
│   |   └─ logging/             # Structured logging
│   |       └─ logger.ts
│   |       └─ request-context.ts
│   |       └─ index.ts
│   |
│   |   └─ validation/          # Validation framework
│   |       └─ validate.ts
│   |       └─ schema-registry.ts
│   |       └─ index.ts
│   |
│   └─ audit/                   # Audit framework
```

```
| | | └─ audit-logger.ts
| | | └─ audit-middleware.ts
| | | └─ audit-models.ts
| | | └─ index.ts
|
| | | └─ events/ # Event Bus
| | |   └─ event-bus.ts
| | |   └─ event-types.ts
| | |   └─ event-handler.ts
| | |   └─ index.ts
|
| | | └─ notifications/ # Notification engine
| | |   └─ notification-engine.ts
| | |   └─ channels/
| | |     └─ in-app.channel.ts
| | |     └─ email.channel.ts
| | |     └─ sms.channel.ts
| | |     └─ whatsapp.channel.ts
| | |   └─ templates/
| | |   └─ subscription-service.ts
| | |   └─ index.ts
|
| | | └─ workflow/ # Workflow engine
| | |   └─ workflow-engine.ts
| | |   └─ state-machine.ts
| | |   └─ transition-guard.ts
| | |   └─ workflow-registry.ts
| | |   └─ decorators.ts
| | |   └─ index.ts
|
| | | └─ permissions/ # RBAC
| | |   └─ permission-middleware.ts
| | |   └─ permission-checker.ts
| | |   └─ permission-decorator.ts
| | |   └─ permission-registry.ts
| | |   └─ index.ts
|
```

```

|   |   |─ tenant/                                # Multi-tenancy
|   |   |   |─ tenant-middleware.ts
|   |   |   |─ tenant-context.ts
|   |   |   └─ index.ts
|   |   |
|   |   |─ transactions/                          # Transaction helper
|   |   |   |─ transaction.ts
|   |   |   └─ index.ts
|   |   |
|   |   |─ files/                                # File upload service
|   |   |   |─ file-service.ts
|   |   |   |─ storage-drivers/
|   |   |   |   |─ s3.driver.ts
|   |   |   |   └─ local.driver.ts
|   |   |   └─ index.ts
|   |   |
|   |   |─ jobs/                                # Background job
framework
|   |   |   |─ job-queue.ts
|   |   |   |─ job-scheduler.ts
|   |   |   |─ job-runner.ts
|   |   |   |─ job-registry.ts
|   |   |   └─ index.ts
|   |   |
|   |   |─ repositories/                        # Base repository
classes
|   |   |   |─ base.repository.ts
|   |   |   |─ soft-delete.repository.ts
|   |   |   |─ tenant.repository.ts
|   |   |   └─ index.ts
|   |   |
|   |   |─ services/                            # Base service classes
|   |   |   |─ base.service.ts
|   |   |   |─ crud.service.ts
|   |   |   └─ index.ts
|   |   |
|   |   |─ controllers/                        # Base controller

```

classes

```
|   |   |   |   |─ base.controller.ts
|   |   |   |   |─ crud.controller.ts
|   |   |   |   |─ index.ts
|   |   |   |   |
|   |   |   |   |─ utils/                                # Shared utilities
|   |   |   |   |   |─ date.ts
|   |   |   |   |   |─ money.ts
|   |   |   |   |   |─ quantity.ts
|   |   |   |   |   |─ crypto.ts
|   |   |   |   |   |─ pagination.ts
|   |   |   |   |   |─ index.ts
|   |   |   |   |
|   |   |   |   |─ modules/                                # Business modules
(one per domain)
|   |   |   |   |─ organization/
|   |   |   |   |   |─ organization.routes.ts
|   |   |   |   |   |─ organization.controller.ts
|   |   |   |   |   |─ organization.service.ts
|   |   |   |   |   |─ organization.repository.ts
|   |   |   |   |   |─ organization.workflow.ts
|   |   |   |   |   |─ organization.events.ts
|   |   |   |   |   |─ organization.schemas.ts
|   |   |   |   |   |─ organization.permissions.ts
|   |   |   |   |   |─ organization.types.ts
|   |   |   |   |   |─ __tests__/
|   |   |   |   |   |   |─ organization.service.test.ts
|   |   |   |   |   |   |─ organization.repository.test.ts
|   |   |   |   |   |   |─ organization.workflow.test.ts
|   |   |   |   |   |   |─ organization.e2e.test.ts
|   |   |   |   |
|   |   |   |   |─ auth/
|   |   |   |   |─ rbac/
|   |   |   |   |─ supplier/
|   |   |   |   |─ product/
|   |   |   |   |─ purchase-order/
|   |   |   |   |─ goods-receipt/
```

```

|   |   └─ iqc/
|   |   └─ quality-hold/
|   |   └─ inventory/
|   |   └─ stock-ledger/
|   |   └─ barcode/
|   |   └─ warehouse/
|   |   └─ ... (future modules follow same structure)
|   |
|   └─ routes/                                # Route registration
|   └─ index.ts                              # Mounts all module
routers
|   └─ health.routes.ts
|   |
|   └─ types/                                # Backend-local types
|       └─ express.d.ts
|
└─ tests/
    └─ unit/
    └─ integration/
    └─ e2e/
    └─ fixtures/
    └─ helpers/
    └─ setup.ts
|
└─ package.json
└─ tsconfig.json
└─ vitest.config.ts
└─ README.md

```

3.3 Frontend Folder Structure (apps/web/)

```

apps/web/
└─ src/
    └─ app/                                # Next.js App Router
    |   └─ layout.tsx
    |   └─ page.tsx                        # Composes module

```

```

shell
├── (auth)/
│   ├── login/page.tsx
│   └── logout/page.tsx
└── (dashboard)/
    ├── layout.tsx # Sidebar + header
shell
├── organization/page.tsx
├── suppliers/page.tsx
├── products/page.tsx
└── ... (one route per module)

└── modules/ # One folder per
business module
├── organization/
│   ├── components/
│   │   ├── OrganizationTree.tsx
│   │   ├── CompanyForm.tsx
│   │   ├── PlantForm.tsx
│   │   └── WarehouseForm.tsx
│   ├── hooks/
│   │   ├── use-organizations.ts
│   │   ├── use-create-company.ts
│   │   └── use-update-plant.ts
│   ├── api/
│   │   └── organization-api.ts
│   ├── types.ts
│   └── index.ts
├── auth/
├── rbac/
├── supplier/
├── product/
└── ... (one per backend module)

└── shared/ # Frontend foundation
    └── api/

```

```

|   |   |   └─ client.ts                # Axios instance +
interceptors
|   |   |   └─ query-client.ts          # React Query client
|   |   |   └─ query-keys.ts            # Centralized query
key factory
|   |   |   └─ auth/
|   |   |   └─ auth-context.tsx
|   |   |   └─ permission-guard.tsx
|   |   |   └─ require-permission.tsx
|   |   |   └─ components/
|   |   |   └─ ui/                      # shadcn/ui base
components
|   |   |   └─ data-table/              # Reusable data table
|   |   |   └─ form/                   # Form helpers
|   |   |   └─ feedback/               # Loading, error,
empty states
|   |   |   └─ layout/
|   |   |   └─ hooks/
|   |   |   └─ use-debounce.ts
|   |   |   └─ use-pagination.ts
|   |   |   └─ use-permission.ts
|   |   |   └─ stores/
|   |   |   └─ auth-store.ts
|   |   |   └─ org-context-store.ts
|   |   |   └─ types/
|   |   |   └─ api-contracts.ts        # Mirrors backend DTOs
|   |   |   └─ utils/
|   |   |
|   |   └─ lib/                        # Next.js lib
(supabase client, etc.)
|
└─ tests/
    └─ unit/
    └─ e2e/                            # Playwright
    └─ fixtures/
|
└─ package.json

```

```

├─ tsconfig.json
├─ vitest.config.ts
├─ playwright.config.ts
└─ README.md

```

3.4 Shared Package (packages/shared/)

```

packages/shared/
├─ src/
│   └─ index.ts                # Barrel export
│   └─ types/
│       └─ brans.ts           # Brand types
│           (TenantId, UserId, etc.)
│       └─ common.ts          # Id, Timestamp, Audit
│       └─ api.ts             # ApiSuccess,
│           ApiError, Paginated
│       └─ enums/
│           └─ status.enum.ts
│           └─ permission.enum.ts
│           └─ entity-type.enum.ts
│           └─ audit-action.enum.ts
│           └─ notification-channel.enum.ts
│           └─ index.ts
│       └─ dtos/
│           └─ pagination.dto.ts
│           └─ filter.dto.ts
│           └─ sort.dto.ts
│           └─ audit-log.dto.ts
│           └─ index.ts
│       └─ events/
│           └─ domain-event.base.ts
│           └─ event-names.ts # Catalog of all event
names
│       └─ constants/
│           └─ regex.ts        # GST, PAN, email,

```



```

phone patterns
|   |   └─ index.ts
|   └─ utils/
|       └─ date.ts
|       └─ money.ts
|       └─ index.ts
└─ package.json
└─ tsconfig.json
└─ README.md

```

Rationale: `packages/shared` is consumed by both `apps/web` and `apps/backend`. This guarantees the frontend and backend always agree on contract shape. Breaking changes here trigger TypeScript errors on both sides.

3.5 Prisma Package (`packages/prisma/`)

```

packages/prisma/
└─ prisma/
|   └─ schema.prisma                # Single source of
truth
|   └─ migrations/                  # One folder per
migration
|       └─ seed/
|           └─ 00-tenants.seed.ts
|           └─ 01-roles-permissions.seed.ts
|           └─ 02-organization.seed.ts
|           └─ 03-products.seed.ts
|           └─ 04-suppliers.seed.ts
|           └─ index.ts
└─ src/
|   └─ client.ts                    # PrismaClient
singleton
|   └─ extensions/                  # Prisma extensions
(sof delete, tenant)
|   |   └─ soft-delete.ts
|   |   └─ tenant-scope.ts

```

```
| | └─ audit-log.ts
| └─ index.ts
└─ package.json
└─ README.md
```

4. Layered Architecture

4.1 Layer Responsibilities

Layer	Responsibility	Cannot Do
Route	Map URL pattern + HTTP method to controller method	Cannot contain logic
Controller	Parse request, validate input shape, call service, shape response	Cannot call Prisma, cannot contain business rules
Service	Business logic, orchestration, workflow invocation, event publishing	Cannot call Prisma directly (must go through repository)
Repository	Data access via Prisma, multi-tenant scoping, soft delete filtering	Cannot contain business rules, cannot call other services
Database	PostgreSQL	—

4.2 Example Flow — “Create Purchase Order”

```
POST /api/purchase-orders
↓
```

```
PurchaseOrderRoute (routes/purchase-order.routes.ts)
  → just maps to controller.create
  ↓
PurchaseOrderController.create
  → validate body (zod schema)
  → extract tenant/user from request context
  → call purchaseOrderService.create(dto, ctx)
  → shape response envelope
  ↓
PurchaseOrderService.create
  → check permission (RBAC)
  → start transaction (transaction helper)
  → validate business rules (supplier active, products
exist, budget available)
  → invoke workflow: poWorkflow.transition('draft',
'submitted', po)
  → call purchaseOrderRepository.create(data) inside
transaction
  → publish event: PurchaseOrderSubmitted
  → commit transaction
  → return created PO
  ↓
PurchaseOrderRepository.create
  → prisma.purchaseOrder.create(...) with tenantId from
context
  → returns domain entity
  ↓
PostgreSQL
```

4.3 Cross-Cutting Concerns (Middleware Order)

Request →

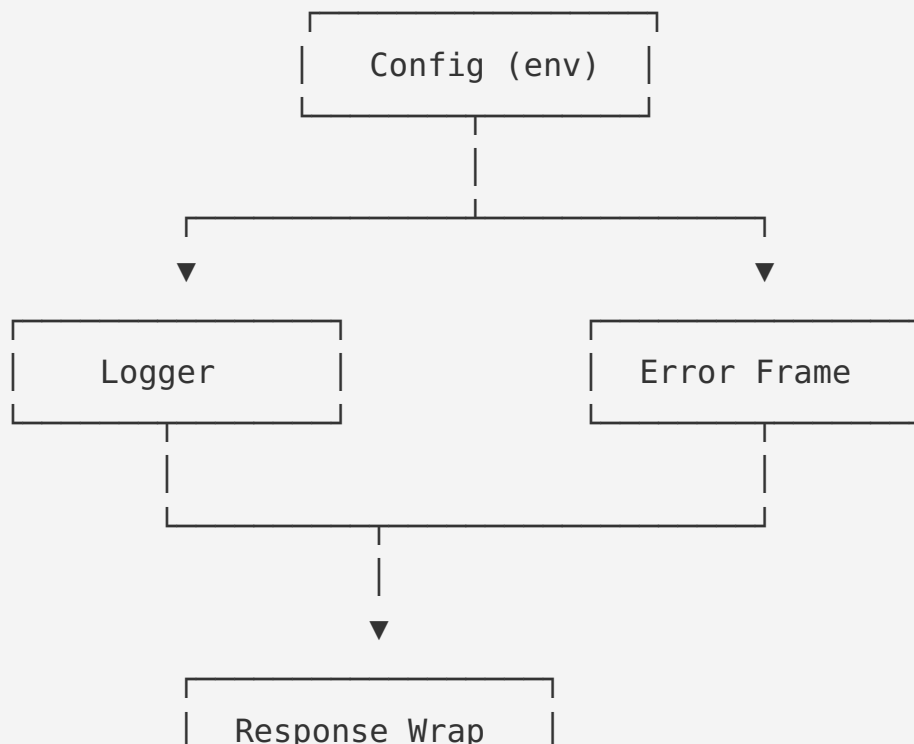
1. RequestIdMiddleware (assigns correlation ID)
2. LoggingMiddleware (logs request start)
3. ErrorBoundaryMiddleware (catches all errors, shapes response)

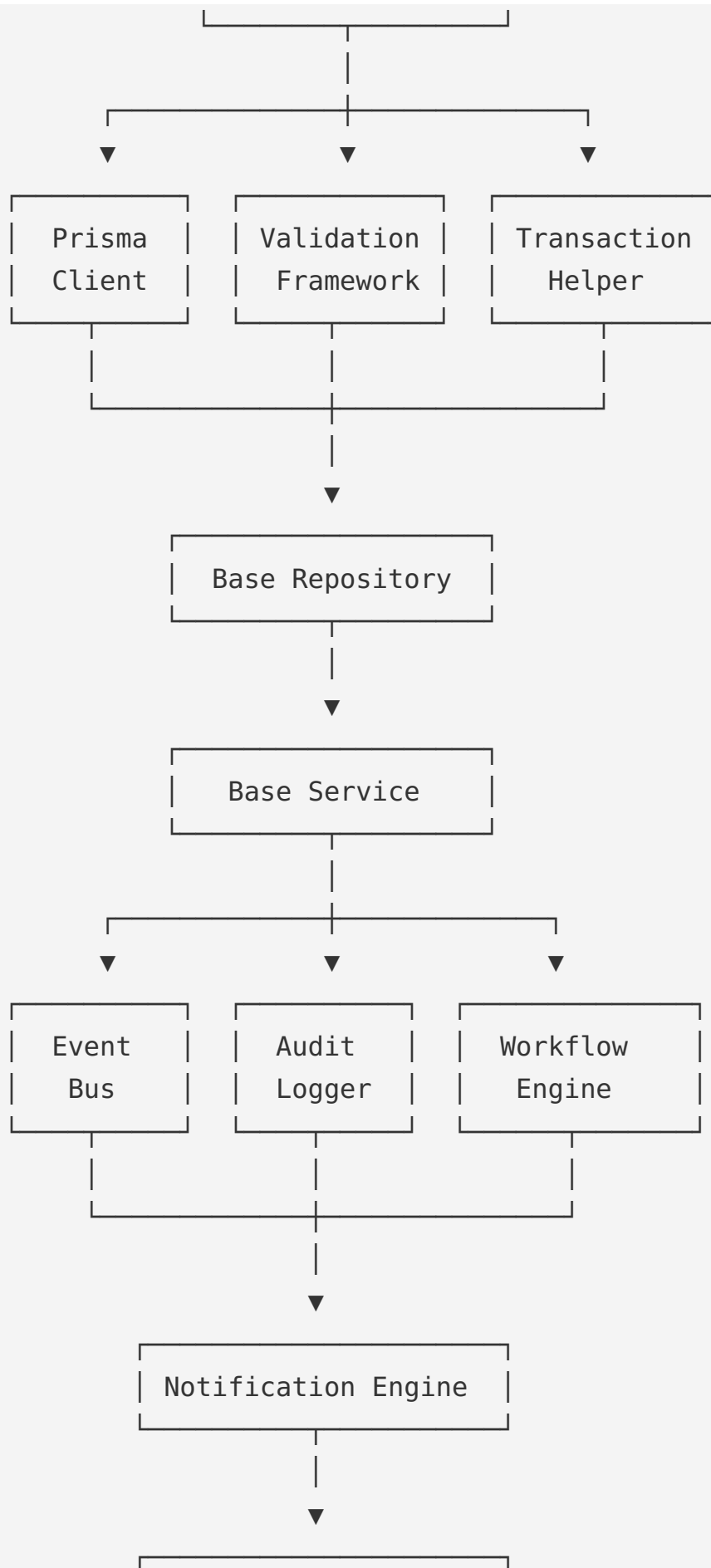
- | | |
|--------------------------|--|
| 4. BodyParserMiddleware | (JSON parsing) |
| 5. AuthMiddleware | (validates JWT, loads user) |
| 6. TenantMiddleware | (loads tenant from JWT or header) |
| 7. RbacMiddleware | (checks route-level permission) |
| 8. AuditMiddleware | (records request metadata for audit log) |
| 9. RateLimitMiddleware | (per-tenant rate limiting) |
| 10. ValidationMiddleware | (per-route zod schema validation) |
| → Handler (Controller) | |

Each middleware runs in order. Any middleware can short-circuit (e.g., Auth fails → 401 returned, no further middleware runs).

5. Dependency Graph

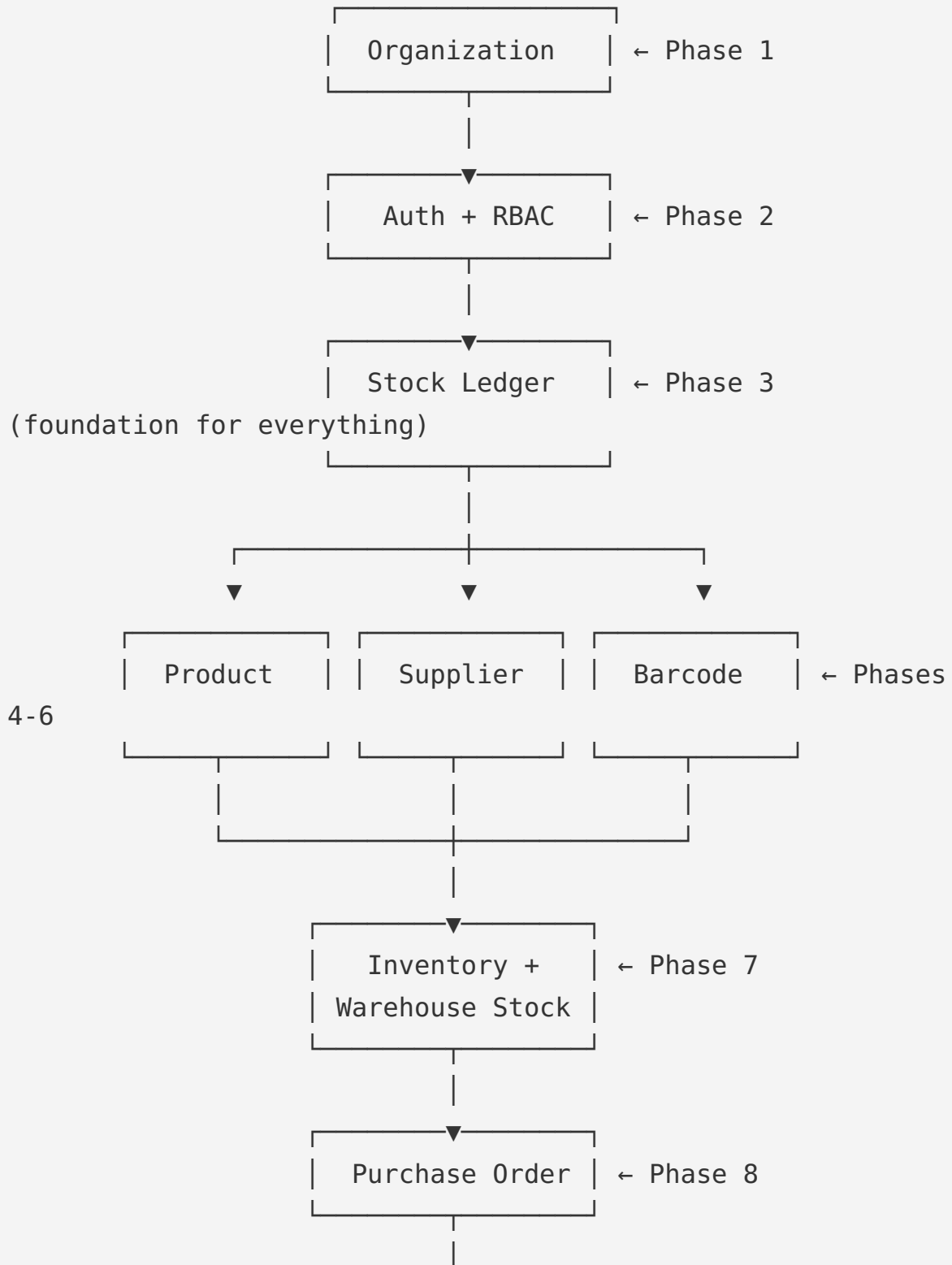
5.1 Foundation Dependency Graph

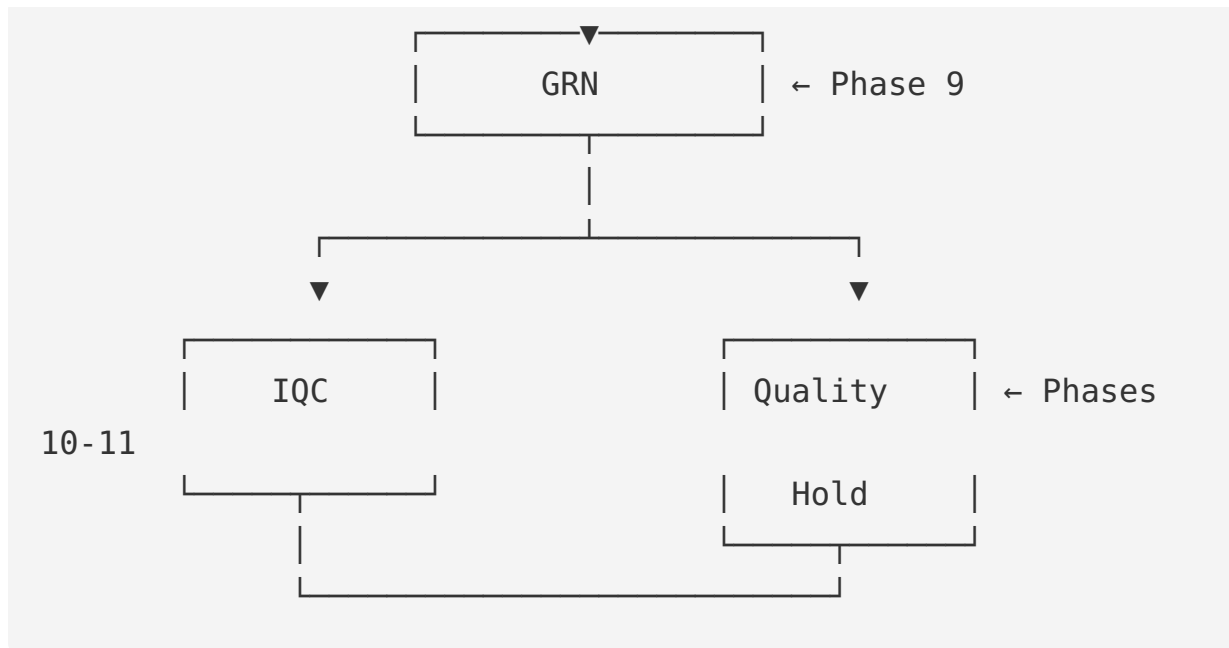




Base Controller

5.2 Module Dependency Graph (for First Journey)





Hard rule: A module may only depend on modules below it in the graph. No upward dependencies. No circular dependencies. Enforced by ESLint `no-restricted-imports` per-module.

6. Shared Libraries

6.1 Library Catalog

Library	Package	Purpose	Consumed By
@suop/shared	packages/ shared	Types, enums, DTOs, events, constants	Backend, Web, Mobile
@suop/prisma	packages/ prisma	PrismaClient, extensions, migrations	Backend only
@suop/ui	packages/ui	Reusable React components (DataTable, Form, etc.)	Web, Mobile (partial)
			All

Library	Package	Purpose	Consumed By
@suop/eslint-config	packages/eslint-config	Shared ESLint rules	

6.2 @suop/shared — Contract Surface

This package exports **only types and constants**. No runtime logic (with rare exceptions like regex patterns and pure functions). This keeps it tree-shakeable and safe to import from both frontend and backend without bundling backend code into the frontend.

Exports: - `Brand<T, B>` — brand types for nominal typing - `EntityId`, `TenantId`, `UserId`, `RoleId`, `ProductId`, etc. — branded strings - `Enum` namespace — all shared enums (see §19) - `DTO` namespace — all shared DTOs (see §20) - `Event` namespace — all domain event names (see §21) - `Regex` — GST, PAN, email, phone patterns - `formatMoney`, `formatDate`, `formatQuantity` — pure formatters

6.3 @suop/prisma — Database Access

Exports: - `prisma` — singleton PrismaClient with extensions registered - `Prisma` — the Prisma namespace (for `Prisma.PromiseReturnType`, etc.) - `withTenant(tenantId)` — helper for cross-tenant queries (rare, audit-logged) - `transaction(fn)` — wraps Prisma `$transaction` with logging + error handling

Prisma extensions registered by default: - **Soft Delete Extension** — overrides `delete` to set `deletedAt`, overrides `findMany/findFirst` to filter `deletedAt IS NULL` by default - **Tenant Scope Extension** — automatically injects `tenantId` from `AsyncLocalStorage` context into all `where` clauses - **Audit Log Extension** — writes to `audit_logs` table on every `create/update/delete`

6.4 @suop/ui — Component Library

Built on top of `shadcn/ui`. Adds enterprise-specific components: - `<DataTable />` — sortable, filterable, paginated table with column definitions - `<EntityForm />` — schema-driven form (zod schema → form

fields) - `<StatusBadge />` — color-coded status pill driven by enum -
`<PermissionGuard>` — hides children if user lacks permission -
`<WorkflowActions />` — renders available actions based on current entity
state - `<AuditTrail />` — renders audit log timeline for an entity -
`<LoadingState />`, `<ErrorState />`, `<EmptyState />` — standard
feedback

7. Base Classes and Interfaces

7.1 BaseRepository<T>

```
abstract class BaseRepository<T extends { id: string;  
    tenantId: string; deletedAt: Date | null; version:  
    number }> {  
    protected constructor(protected readonly prisma:  
        PrismaClient, protected readonly modelName:  
        Prisma.ModelName) {}  
  
    async findById(ctx: TenantContext, id: string): Promise<T  
        | null>  
    async findByIdOrThrow(ctx: TenantContext, id: string):  
        Promise<T> // throws NotFoundError  
    async list(ctx: TenantContext, query: ListQuery):  
        Promise<PaginatedResult<T>>  
    async create(ctx: RequestContext, data: CreatedDto<T>):  
        Promise<T>  
    async update(ctx: RequestContext, id: string, data:  
        UpdatedDto<T>, expectedVersion: number): Promise<T>  
    async softDelete(ctx: RequestContext, id: string,  
        expectedVersion: number): Promise<void>  
    async count(ctx: TenantContext, filter?: FilterDto<T>):  
        Promise<number>  
    async exists(ctx: TenantContext, filter: FilterDto<T>):  
        Promise<boolean>  
}
```

Features: - Multi-tenant scoping automatic (via Prisma extension) - Soft
delete filter automatic - Optimistic locking on `update` and `softDelete`
(throws `ConcurrencyError` on version mismatch) - `RequestContext`

carries `userId`, `tenantId`, `correlationId`, `ip`, `userAgent` for audit -
`ListQuery` supports pagination, sorting, filtering, field selection

7.2 `BaseService<TEntity, TRepository>`

```
abstract class BaseService<TEntity, TRepository> extends
    BaseRepository<TEntity>> {
    protected constructor(protected readonly repo:
        TRepository) {}

    async get(ctx: RequestContext, id: string):
        Promise<TEntity>
    async list(ctx: RequestContext, query: ListQuery):
        Promise<PaginatedResult<TEntity>>
    async create(ctx: RequestContext, dto:
        CreatedDto<TEntity>): Promise<TEntity>
    async update(ctx: RequestContext, id: string, dto:
        UpdatedDto<TEntity>, version: number):
        Promise<TEntity>
    async delete(ctx: RequestContext, id: string, version:
        number): Promise<void>

    // Hooks for subclasses to override
    protected async beforeCreate(ctx: RequestContext, dto:
        CreatedDto<TEntity>): Promise<CreatedDto<TEntity>> {
        return dto }
    protected async afterCreate(ctx: RequestContext, entity:
        TEntity): Promise<void> {}
    protected async beforeUpdate(ctx: RequestContext, entity:
        TEntity, dto: UpdatedDto<TEntity>):
        Promise<UpdatedDto<TEntity>> { return dto }
    protected async afterUpdate(ctx: RequestContext, before:
        TEntity, after: TEntity): Promise<void> {}
    protected async beforeDelete(ctx: RequestContext, entity:
        TEntity): Promise<void> {}
    protected async afterDelete(ctx: RequestContext, entity:
        TEntity): Promise<void> {}

    // Validation hook – subclasses implement
    protected abstract validateCreate(ctx: RequestContext,
        dto: CreatedDto<TEntity>): Promise<ValidationResult>
```

```

protected abstract validateUpdate(ctx: RequestContext,
    entity: TEntity, dto: UpdateDto<TEntity>):
    Promise<ValidationResult>
}

```

Features: - `before*` / `after*` hooks for cross-cutting logic without subclassing for every concern - `validateCreate` / `validateUpdate` are abstract — every concrete service must implement - All mutations wrapped in transactions automatically (override `create` / `update` if transaction boundary needs to span multiple operations)

7.3 BaseController<TEntity>

```

abstract class BaseController<TEntity> {
    // Standard CRUD handlers – subclasses register these on routes
    protected async handleGet(ctx: RequestContext):
        Promise<ResponseEnvelope<TEntity>>
    protected async handleList(ctx: RequestContext):
        Promise<ResponseEnvelope<PaginatedResult<TEntity>>>
    protected async handleCreate(ctx: RequestContext):
        Promise<ResponseEnvelope<TEntity>>
    protected async handleUpdate(ctx: RequestContext):
        Promise<ResponseEnvelope<TEntity>>
    protected async handleDelete(ctx: RequestContext):
        Promise<ResponseEnvelope<{ deleted: boolean }>>
}

```

Features: - Controllers are thin — they delegate to services - All responses wrapped in `ResponseEnvelope` (see §13) - All errors thrown by services propagate to `ErrorBoundaryMiddleware` which shapes them - Input validation via zod schema attached to route definition

7.4 BaseWorkflow<TState extends string, TContext>

```

abstract class BaseWorkflow<TState extends string, TContext
    extends { id: string; status: TState; version:
        number }> {
    protected abstract readonly states: TState[]
}

```

```

protected abstract readonly transitions:
    Transition<TState, TContext>[]
protected abstract readonly initialState: TState

async canTransition(ctx: RequestContext, entity:
    TContext, target: TState): Promise<boolean>
async transition(ctx: RequestContext, entity: TContext,
    target: TState, payload?: unknown):
    Promise<TContext>
protected abstract onBeforeTransition(ctx:
    RequestContext, entity: TContext, target: TState,
    payload?: unknown): Promise<void>
protected abstract onAfterTransition(ctx: RequestContext,
    before: TContext, after: TContext, payload?:
    unknown): Promise<void>
}

```

Features: - States and transitions declared declaratively - `canTransition` checks guards (sync + async) - `transition` validates, runs pre-hooks, persists, runs post-hooks, publishes event, writes audit log — all in a transaction - Pre/post hooks let subclasses add behavior without re-implementing the engine

7.5 BaseEventHandler<TEvent extends DomainEvent>

```

abstract class BaseEventHandler<TEvent extends DomainEvent>
{
    abstract readonly eventName: string
    abstract handle(event: TEvent, ctx: EventHandlerContext):
        Promise<void>
    // Optional retry policy
    retries?: number = 3
    backoffMs?: number = 1000
}

```

7.6 BaseJob

```

abstract class BaseJob<TPayload> {
    abstract readonly jobName: string
}

```

```
abstract readonly concurrency: number // max parallel
    instances

abstract readonly maxRetries: number
abstract execute(payload: TPayload, ctx: JobContext):
    Promise<JobResult>
}
```

7.7 BaseMiddleware

```
abstract class BaseMiddleware {
    abstract handle(ctx: RequestContext, next: () =>
        Promise<void>): Promise<void>
}
```

8. Middleware Stack

8.1 RequestIdMiddleware

- Generates UUID v7 for every request
- Stores in `AsyncLocalStorage` for access by logger, audit, etc.
- Echoes back in `X-Request-Id` response header
- If incoming request has `X-Request-Id`, uses that (for distributed tracing)

8.2 LoggingMiddleware

- Logs request start: method, path, userId (if authenticated), tenantId, requestBody (sanitized)
- Logs request end: status, duration, responseSize
- Uses structured JSON logging (Pino)
- Redacts sensitive fields (`password`, `token`, `auth`, `creditCard`)
- Logs at `info` for 2xx, `warn` for 4xx, `error` for 5xx

8.3 ErrorBoundaryMiddleware

- Wraps all downstream middleware in try/catch

- Catches `BaseError` subclasses → shaped response with correct status code
- Catches unknown errors → 500 with generic message, full stack logged
- Never leaks stack traces to client in production
- Publishes `SystemError` event for monitoring

8.4 AuthMiddleware

- Extracts JWT from `Authorization: Bearer <token>` header
- Validates signature, expiry, issuer, audience
- Loads user from DB (cached 60s) — checks `status === 'ACTIVE'`
- Loads tenant from JWT claim
- Sets `RequestContext.user` and `RequestContext.tenant`
- Public routes (`/api/auth/login` , `/api/health`) bypass this middleware
- Missing/invalid token → 401 with `WWW-Authenticate: Bearer` header

8.5 TenantMiddleware

- Reads `tenantId` from `RequestContext` (set by `AuthMiddleware`)
- If absent (e.g., super-admin), validates user has `system:tenant:cross` permission
- Sets up `AsyncLocalStorage` tenant context — all Prisma queries auto-scope
- Detects tenant mismatch (user from tenant A trying to access tenant B's data) → 403

8.6 RbacMiddleware

- Route declares required permission: `router.post('/purchase-orders', requirePermission('po:create'), ...)`
- Middleware reads user's roles from `RequestContext`
- Checks if any role grants the required permission
- Optional attribute-based check: e.g., user can only approve POs they created (policy engine)
- 403 if denied; audit log entry written at `WARN` severity

8.7 AuditMiddleware

- Records request metadata for every mutating request (POST/PUT/PATCH/DELETE)
- Captures: timestamp, userId, tenantId, action (HTTP method), entity type (from route), entity ID (from path/body), before/after (from service layer)
- Writes to `audit_logs` table in a non-blocking fire-and-forget manner
- For high-severity actions (delete, permission change, blacklist, recall), also publishes `HighSeverityAudit` event for real-time alerting

8.8 RateLimitMiddleware

- Per-tenant + per-IP rate limiting
- Configurable per route (e.g., `/api/auth/login` allows 10/min/IP; `/api/purchase-orders` allows 100/min/tenant)
- Uses Redis sliding window
- 429 response with `Retry-After` header when exceeded

8.9 ValidationMiddleware

- Route attaches zod schema: `router.post('/purchase-orders', validateBody(createPoSchema), ...)`
- Middleware parses body, validates against schema
- On failure → 400 with structured validation errors (field path, message, code)
- Schema is the single source of truth — same schema is used by frontend for client-side validation

9. Workflow Engine

9.1 Purpose

Every entity with a lifecycle must declare a state machine. The engine enforces:

- Only valid transitions are allowed
- Guards (pre-conditions) are checked before transition
- Side effects (hooks) run before and after

transition - Domain events are published on every transition - Audit log entry is written on every transition - All of the above happens in a single database transaction

9.2 State Machine Definition Example (Purchase Order)

```
// modules/purchase-order/purchase-order.workflow.ts
export class PurchaseOrderWorkflow extends
    BaseWorkflow<PoStatus, PurchaseOrderContext> {
    readonly initialState = 'DRAFT'
    readonly states = ['DRAFT', 'SUBMITTED', 'APPROVED',
        'ORDERED', 'PARTIALLY_RECEIVED', 'RECEIVED',
        'CLOSED', 'CANCELLED'] as const

    readonly transitions: Transition<PoStatus,
        PurchaseOrderContext>[] = [
        { from: 'DRAFT', to: 'SUBMITTED', guard: canSubmit,
            onAfter: notifyApprovers },
        { from: 'SUBMITTED', to: 'APPROVED', guard: canApprove,
            onAfter: notifySupplier },
        { from: 'SUBMITTED', to: 'DRAFT', guard: canReject,
            onAfter: notifyRequester },
        { from: 'APPROVED', to: 'ORDERED', onAfter:
            sendPOToSupplier },
        { from: 'ORDERED', to: 'PARTIALLY_RECEIVED', onAfter:
            notifyProcurement },
        { from: 'ORDERED', to: 'RECEIVED', onAfter:
            triggerGrnClosure },
        { from: 'PARTIALLY_RECEIVED', to: 'RECEIVED', onAfter:
            triggerGrnClosure },
        { from: 'RECEIVED', to: 'CLOSED', guard:
            allLinesMatchedInvoiced },
        { from: 'DRAFT', to: 'CANCELLED', onAfter:
            notifyRequester },
        { from: 'SUBMITTED', to: 'CANCELLED', guard: canCancel,
            onAfter: notifyRequester },
        { from: 'APPROVED', to: 'CANCELLED', guard:
            canCancelApproved, onAfter:
            notifySupplierAndRequester },
    ]
}
```



```

protected async onBeforeTransition(ctx, entity, target,
    payload) {
    // Re-validate entity is still in expected state
    // (optimistic concurrency)
    const fresh = await this.repo.findByIdOrThrow(ctx,
        entity.id)
    if (fresh.status !== entity.status) throw new
        ConcurrencyError('PO state changed')
}

protected async onAfterTransition(ctx, before, after,
    payload) {
    // Audit log
    await this.auditLogger.log({
        action: 'WORKFLOW_TRANSITION',
        entityType: 'PurchaseOrder',
        entityId: after.id,
        fromState: before.status,
        toState: after.status,
        actorId: ctx.userId,
        tenantId: ctx.tenantId,
    })
    // Domain event
    await this.eventBus.publish({
        name: 'PurchaseOrderTransitioned',
        payload: { poId: after.id, from: before.status, to:
            after.status, actorId: ctx.userId },
    })
}
}

```

9.3 Transition Guard Contract

A guard is an async predicate that receives `(ctx, entity, payload)` and returns `boolean | { allowed: false, reason: string }`. Guards must be **side-effect free** — they only check whether a transition is allowed.

9.4 Workflow Registry

Every workflow is registered in `WorkflowRegistry` at boot. The registry: - Validates no duplicate state machines for the same entity type - Provides `registry.getWorkflowFor('PurchaseOrder')` lookup - Exposes metadata for UI: “given current state, what actions are available?” — used by `<WorkflowActions />` frontend component

9.5 Workflow Persistence

State is stored on the entity itself (`status` field). The workflow engine does not maintain a separate state store — it operates on the entity’s `status` field, ensuring the DB is always the source of truth. This avoids drift between workflow state and entity state.

9.6 Workflow Inspection API

`GET /api/workflows/:entityType/:entityId` returns: - Current state - Available transitions (with guard results) - Transition history (last 20 transitions from audit log)

This powers the UI’s workflow visualization and action buttons.

10. Notification Engine

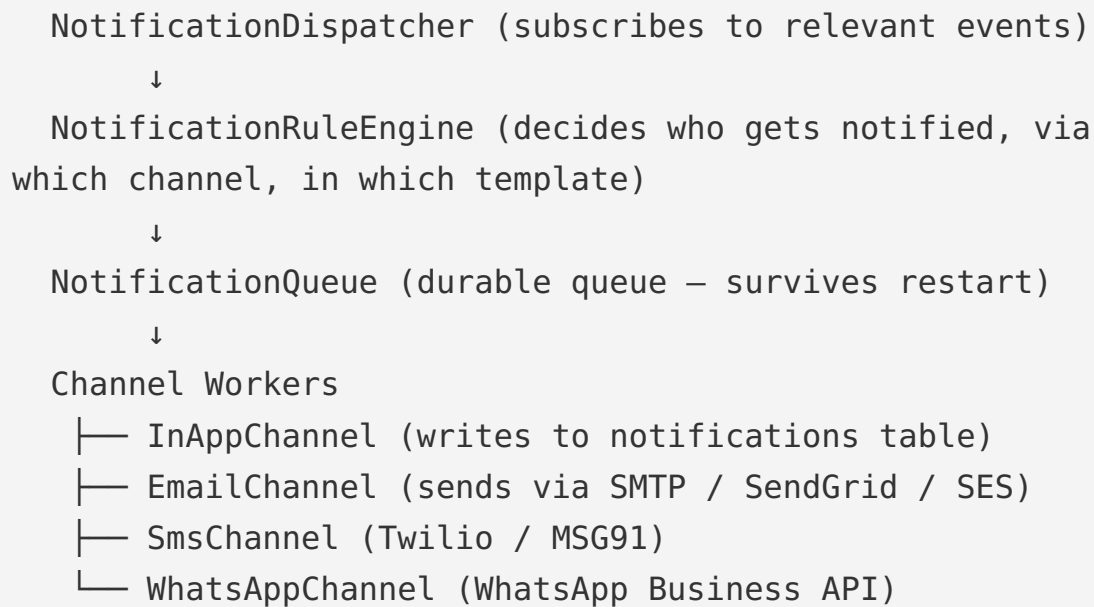
10.1 Purpose

Decouple “things that happen” from “people who need to know about them.” Business logic publishes domain events; the notification engine subscribes to events and routes notifications to the right users via the right channels.

10.2 Architecture

Domain Event Bus





10.3 Notification Rule Configuration

Rules are stored in DB and configurable per tenant:

```
interface NotificationRule {
  id: string
  tenantId: string
  eventName: string // e.g.,
    'PurchaseOrderSubmitted'
  recipientResolver: 'APPROVERS' | 'REQUESTER' |
    'ROLE:PLANT_HEAD' | 'USER' | 'CUSTOM'
  recipientConfig: object // depends on resolver
    - e.g., { roleId: '...' }
  channels: ('IN_APP' | 'EMAIL' | 'SMS' | 'WHATSAPP')[]
  templateId: string
  priority: 'LOW' | 'NORMAL' | 'HIGH' | 'CRITICAL'
  enabled: boolean
}
```

10.4 Templates

Templates use Handlebars syntax with typed contexts:

```

interface EmailTemplate<TContext> {
  id: string
  name: string
  subject: string                                // Handlebars
  htmlBody: string                               // Handlebars
  textBody: string                                // Handlebars
              (fallback)
  contextSchema: z.ZodSchema<TContext>
}

```

Templates stored in DB, version-controlled. Editing a template creates a new version; old versions retained for audit.

10.5 Subscription Service

Users can subscribe/unsubscribe from notification types (e.g., “I want PO approval emails but not SMS”). Per-user preference overrides rule defaults for non-critical notifications. Critical notifications (food safety recall, security alert) cannot be unsubscribed.

10.6 Durable Delivery

- Notifications written to `notification_outbox` table inside the same transaction as the business event
- Channel workers poll the outbox and deliver
- Retry with exponential backoff (3 attempts, then dead-letter queue)
- Delivery status tracked: `PENDING` → `SENT` → `DELIVERED` / `FAILED`
- Audit log records every delivery attempt

10.7 Initial Channels

- **In-App**: writes to `notifications` table; frontend polls every 30s via React Query; future: WebSocket push
 - **Email**: SMTP via Nodemailer (dev) / SES (prod)
 - **SMS**: Twilio (configurable)
 - **WhatsApp**: WhatsApp Business Cloud API (configurable, future)
-

11. Audit Framework

11.1 Purpose

Every mutation (create, update, delete, workflow transition, approval, permission change) must produce an immutable audit log entry. The audit log is the legal record of system activity and must survive even if the underlying business record is deleted.

11.2 Audit Log Schema (Conceptual)

```
interface AuditLog {
  id: string // UUID v7 (sortable)
  timestamp: Date
  tenantId: string
  // Actor
  actorType: 'USER' | 'SYSTEM' | 'API_KEY' | 'JOB'
  actorId: string | null
  actorName: string | null
  actorRole: string | null
  ipAddress: string | null
  userAgent: string | null
  correlationId: string // request ID
  // Action
  action: AuditAction // CREATE, UPDATE,
  // DELETE, APPROVE, TRANSITION, LOGIN, etc.
  severity: 'INFO' | 'WARN' | 'CRITICAL'
  // Entity
  entityType: string // 'PurchaseOrder',
  // 'User', etc.
  entityId: string | null
  entityCode: string | null // human-readable:
  // 'PO-2026-000001'
  // Change
  before: object | null // JSON snapshot
  after: object | null // JSON snapshot
  diff: object | null // field-level diff
```

```
// Context
reason: string |
    null           // user-supplied reason for
    action
metadata: object   // workflow transition
    info, etc.
}
```

11.3 Immutability

- Audit log table is **append-only**. No UPDATE, no DELETE.
- Enforced at database level via PostgreSQL row-level security policy:
`FOR INSERT ONLY`
- The `audit_logs` table is excluded from soft-delete extension
- Backup strategy: audit logs are exported daily to cold storage (S3 Glacier) and never deleted

11.4 Automatic vs Manual Logging

Automatic (via AuditMiddleware + Prisma extension): - Every CRUD mutation - Every workflow transition - Every login/logout - Every permission check failure

Manual (via `auditLogger.log()` from services): - Business events that don't change data (e.g., "PO sent to supplier via email") - High-severity events (e.g., "Critical NCR auto-quarantined batch X") - Cross-cutting events that the middleware can't detect

11.5 Audit Query API

- `GET /api/audit/entity/:entityType/:entityId` — full history of one entity
- `GET /api/audit/actor/:userId` — all actions by one user
- `GET /api/audit/timeline?`
`from=...&to=...&action=...&severity=...` — filtered timeline
- All queries require `audit:read` permission
- Critical-severity audit logs require `audit:read:critical` permission

11.6 Audit Trail UI

The `<AuditTrail entityId=... entityType=... />` component renders a reverse-chronological timeline with: - Timestamp (relative + absolute) - Actor name + role - Action + state transition arrow - Field-level diff (collapsible) - Reason / notes

12. Validation Framework

12.1 Two Layers of Validation

Layer 1 — Schema Validation (zod): - Validates request shape (body, params, query) - Run by `ValidationMiddleware` before controller executes - Schema is co-located with the route definition - On failure: 400 with structured error
(`{ field: 'poLines.0.quantity', message: 'Must be positive', code: 'INVALID' }`) - Same zod schema is exported to frontend for client-side pre-validation

Layer 2 — Business Rule Validation (Service): - Validates business rules that span multiple entities - Run inside service before mutation - Examples: - “Supplier must be ACTIVE to add to PO” - “Total PO value cannot exceed department budget” - “Cannot close PO with un-received lines” - On failure: throws `BusinessRuleError` with `code`, `message`, `entityId` — caught by `ErrorBoundaryMiddleware` → 422 response

12.2 Schema Registry

Every zod schema is registered in `SchemaRegistry` at boot: - Validates no duplicate schema names - Exposes `GET /api/_internal/schemas/:name` endpoint — frontend fetches schema at build time to generate TypeScript types and form fields - Schemas are versioned (breaking changes require new schema name or version suffix)

12.3 Reusable Schema Fragments

Common patterns are extracted to shared schema fragments: - `schemas/identifier.ts` — UUID v7 validation - `schemas/money.ts` — `Decimal(14, 2)` with non-negative constraint - `schemas/quantity.ts` — `Decimal(14, 3)` with non-negative constraint - `schemas/gst.ts` — Indian GST number format - `schemas/pan.ts` — Indian PAN format - `schemas/pagination.ts` — `page`, `pageSize`, `sort`, `filter` parameters - `schemas/audit.ts` — `reason`, `comment`, `notes`

12.4 Validation Result Shape

```
type ValidationResult =
  | { valid: true }
  | { valid: false; errors: ValidationError[] }

interface ValidationError {
  field: string // dot-path:
    'poLines.0.quantity'
  message: string // human-readable
  code: string // 'REQUIRED' |
    'INVALID_FORMAT' | 'BUSINESS_RULE' | 'CONCURRENCY'
  metadata?: object // e.g., { min: 0,
    actual: -5 }
}
```

13. Error Framework

13.1 Error Hierarchy

```
BaseError (abstract)
├─ ValidationError (400)
│   └─ FieldValidationError
├─ AuthenticationError (401)
│   └─ TokenExpiredError
```



```

└─ AuthorizationError (403)
  └─ PermissionDeniedError
└─ NotFoundError (404)
└─ ConcurrencyError (409)          // optimistic lock
conflict
└─ ConflictError (409)            // duplicate / state
conflict
└─ BusinessRuleError (422)        // business rule
violated
└─ RateLimitError (429)
└─ ExternalServiceError (502)     // upstream service
failure
└─ DatabaseError (500)
└─ InternalError (500)

```

13.2 Error Code Catalog

Every error type has a stable, documented code. Codes are namespaced by domain:

```

VALIDATION.REQUIRED_FIELD
VALIDATION.INVALID_FORMAT
VALIDATION.BUSINESS_RULE
AUTH.TOKEN_EXPIRED
AUTH.INVALID_CREDENTIALS
RBAC.PERMISSION_DENIED
PO.BUDGET_EXCEEDED
PO.SUPPLIER_INACTIVE
GRN.QUANTITY_EXCEEDS_PO
STOCK.INSUFFICIENT_STOCK
STOCK.NEGATIVE_BALANCE
CONCURRENCY.VERSION_MISMATCH
...

```

Codes are catalogued in `packages/shared/src/enums/error-code.enum.ts`. Frontend uses codes to render localized messages.

13.3 Response Envelope

Every API response (success or error) uses the same envelope:

```
// Success
{
  success: true,
  data: T,
  message?: string,           // human-readable
                             success message
  meta?: {                   // pagination, etc.
    page: number,
    pageSize: number,
    total: number,
    correlationId: string,
  },
}

// Error
{
  success: false,
  error: {
    code: string,             //
    'PO.SUPPLIER_INACTIVE'
    message: string,          // 'Supplier SUP-001
    is not active'
    details?: ValidationErrors[], // for validation
    errors
    entityId?:
      string,                 // for entity-specific
      errors
    retryAfter?: number,      // for rate limit
  },
  meta: {
    correlationId: string,
  },
}
```

13.4 Error Logging

- 4xx errors logged at `WARN` level
- 5xx errors logged at `ERROR` level with full stack trace
- All errors correlated by `correlationId` from request context
- 5xx errors published to `SystemError` event bus → notification engine alerts engineering channel (Slack/Teams)

13.5 Frontend Error Handling

- Axios interceptor unwraps `ResponseEnvelope`
- On error, interceptor throws typed `ApiError` with code + message
- React Query `onError` handler shows toast for transient errors, redirects to login for 401, shows full-page error for 5xx
- Validation errors are mapped to form fields via react-hook-form

14. Configuration Framework

14.1 Environment Variable Validation

All env vars validated at boot via zod. App refuses to start if validation fails.

```
// config/env.ts
const envSchema = z.object({
  NODE_ENV: z.enum(['development', 'staging', 'production']),
  PORT: z.coerce.number().default(3030),

  DATABASE_URL: z.string().url(),
  DATABASE_POOL_SIZE: z.coerce.number().default(10),

  JWT_SECRET: z.string().min(32),
  JWT_ISSUER: z.string().default('suop-erp'),
  JWT_ACCESS_TTL_MIN: z.coerce.number().default(15),
  JWT_REFRESH_TTL_DAYS: z.coerce.number().default(30),
```

```

    REDIS_URL: z.string().url(),

    S3_ENDPOINT: z.string().url(),
    S3_BUCKET: z.string(),
    S3_ACCESS_KEY: z.string(),
    S3_SECRET_KEY: z.string(),

    SMTP_HOST: z.string(),
    SMTP_PORT: z.coerce.number(),
    SMTP_USER: z.string(),
    SMTP_PASS: z.string(),

    TWILIO_ACCOUNT_SID: z.string().optional(),
    TWILIO_AUTH_TOKEN: z.string().optional(),

    SENTRY_DSN: z.string().url().optional(),
  })

  export const env = envSchema.parse(process.env)

```

14.2 Feature Flags

Feature flags stored in DB, cached in memory, refreshable via admin API.
 Use cases: - Roll out new module to specific tenants - Disable broken feature without deploy - A/B test workflow changes

```

// config/features.ts
class FeatureFlags {
  async isEnabled(tenantId: string, flagName: string):
    Promise<boolean>
  async setFlag(tenantId: string, flagName: string,
    enabled: boolean, reason: string): Promise<void>
  async listFlags(tenantId: string): Promise<FeatureFlag[]>
}

```

Flags checked in code via:

```
if (await features.isEnabled(ctx.tenantId,
    'NEW_RECALL_ENGINE')) {
    // use new engine
} else {
    // use legacy
}
```

14.3 Configuration Sources (in priority order)

1. Environment variables (highest priority — for secrets + deployment-specific)
2. Database (feature flags, per-tenant config)
3. Default values (lowest priority)

15. Transaction Helper

15.1 Purpose

Database transactions must be: - Explicit (no auto-commit magic) - Logged (long-running transactions warned) - Retryable (deadlock + serialization failure → retry with backoff) - Nestable-aware (Prisma interactive transactions don't support nesting — detect and throw clear error)

15.2 API

```
// transactions/transaction.ts
async function transaction<T>(
  ctx: RequestContext,
  fn: (tx: PrismaTransaction) => Promise<T>,
  options?: { isolationLevel?: 'READ_COMMITTED' |
    'SERIALIZABLE'; maxRetries?: number; timeoutMs?:
    number }
): Promise<T>
```

15.3 Retry Policy

- Retry on `P2034` (transaction deadlocked) and `P2038` (serialization failure)
- Exponential backoff: 50ms, 100ms, 200ms, 400ms (max 4 attempts)
- On final failure: throws `DatabaseError` with `code: 'TXN.RETRY_EXHAUSTED'`

15.4 Transaction Context

Inside a transaction, the `RequestContext` carries the `PrismaTransaction` client. All repository methods accept `ctx` and use `ctx.db` (which is either the singleton Prisma client or the transaction client) — so repositories don't need to know they're inside a transaction.

15.5 Long Transaction Detection

Transactions exceeding 5 seconds log a `WARN`. Transactions exceeding 30 seconds log an `ERROR` and publish `SlowTransaction` event. Configurable per-route.

15.6 Outbox Pattern

For mutations that also need to publish domain events, the outbox pattern is used:

1. Inside transaction:
 - Mutate business tables
 - Write event to `event_outbox` table (same transaction)
 2. After commit:
 - Background job reads from `event_outbox`, publishes to event bus, marks as published
 3. Guarantees at-least-once delivery of events even if the bus is temporarily unavailable
-

16. File Upload Service

16.1 Purpose

Unified file storage abstraction for evidence attachments, COA PDFs, calibration certs, audit reports, profile photos, etc.

16.2 API

```
interface FileService {
  async upload(ctx: RequestContext, file: UploadInput):
    Promise<UploadedFile>
  async download(ctx: RequestContext, fileId: string):
    Promise<DownloadedFile>
  async delete(ctx: RequestContext, fileId: string):
    Promise<void>
  async getSignedUrl(ctx: RequestContext, fileId: string,
    ttlSeconds: number): Promise<string>
}

interface UploadInput {
  filename: string
  mimeType: string
  size: number
  stream: ReadableStream           // or Buffer
  category:
    FileCategory                   // COA, EVIDENCE,
    CALIBRATION_CERT, etc.
  linkedEntityType?: string
  linkedEntityId?: string
}

interface UploadedFile {
  id: string
  filename: string
  mimeType: string
  size: number
```

```
storageKey: string           // S3 key or
                        local path
url: string                  // signed URL
                        (short-lived)
checksum: string            // SHA-256
uploadedAt: Date
uploadedBy: string
}
```

16.3 Storage Drivers

- **S3Driver** (default): S3 / MinIO / Supabase Storage (S3-compatible)
- **LocalDriver** (dev only): writes to local filesystem — never used in production

Driver selected via `FILE_STORAGE_DRIVER` env var. CI fails if production build uses LocalDriver.

16.4 File Metadata Persistence

File metadata stored in `file_uploads` table. The actual file blob is in object storage. The DB record tracks: - `id`, `tenantId`, `filename`, `mimeType`, `size`, `checksum` - `storageKey` (S3 key), `storageDriver` - `category`, `linkedEntityType`, `linkedEntityId` - `uploadedBy`, `uploadedAt`, `deletedAt` (soft delete)

16.5 Security

- All uploads require authentication
- File size limits per category (e.g., evidence photos max 10MB, COA PDFs max 5MB)
- MIME type whitelist per category
- Virus scan via ClamAV (async, blocks download if scan pending)
- Signed URLs short-lived (default 15 min) — never expose direct S3 URLs to clients
- File deletion soft-deletes metadata; actual blob deleted after retention period (default 90 days)

16.6 Image Processing (Future)

For images, automatic thumbnail generation (100px, 500px, original) via Sharp. EXIF metadata stripped. Optional watermarking.

17. Background Job Framework

17.1 Purpose

Long-running or scheduled work must run outside the request/response cycle. Examples: - Escalation checker (every 5 min): find overdue CAPAs, escalate - Notification dispatcher (every 30s): drain notification outbox - Audit log archival (daily): export audit logs to cold storage - Stock reorder check (hourly): find products below reorder level - Mock recall drill (scheduled): run mock recall exercise

17.2 Job Types

Scheduled Jobs (cron): - Defined via cron expression - Single instance per tenant (distributed lock via Redis prevents duplicates) - Example:

```
escalation-checker: '* /5 * * * *
```

Queued Jobs: - Triggered by application code:

```
jobs.enqueue('SendWelcomeEmail', { userId })
```

 - Processed by workers - Retry with exponential backoff - Example: user registers → enqueue welcome email job

17.3 Job Queue Implementation

- BullMQ on Redis (primary)
- In-memory fallback for development
- Job payload stored as JSON; large payloads (>32KB) stored in S3, payload references the S3 key

17.4 Job Lifecycle

```
ENQUEUED → RUNNING → COMPLETED
    ↘ FAILED (retries remaining)
        ↘ DEAD_LETTER_QUEUE (max retries
exhausted)
```

17.5 Job Observability

- `GET /api/_internal/jobs` — list jobs with filters (status, name, tenant)
- `GET /api/_internal/jobs/:id` — job detail with retry history
- `POST /api/_internal/jobs/:id/retry` — manually retry dead-lettered job (requires admin)
- Failed jobs publish `JobFailed` event → notification engine alerts on-call

17.6 Job Definition Contract

Every job class extends `BaseJob<TPayload>` : - `jobName` — unique identifier - `concurrency` — max parallel instances per worker (default 1) - `maxRetries` — default 3 - `execute(payload, ctx)` — actual work

Jobs registered in `JobRegistry` at boot. Registry validates no duplicate names.

18. Event Bus

18.1 Purpose

Decouple modules via in-process pub/sub. When a Purchase Order is submitted, the PO module publishes `PurchaseOrderSubmitted`. The notification engine, the audit framework, and any future subscriber react independently. The PO module knows nothing about who's listening.

18.2 In-Process vs Distributed

- **Phase 0 (now):** In-process event bus using `EventEmitter3` + `AsyncLocalStorage`. Subscribers run in the same process. Sufficient for single-instance deployment.
- **Future (when scaling):** Migrate to Redis Pub/Sub or Kafka for multi-instance. The `EventBus` interface stays the same; only the implementation changes.

18.3 Event Bus API

```
interface EventBus {
  async publish<T extends DomainEvent>(event: T):
    Promise<void>
  subscribe<T extends DomainEvent>(eventName: string,
    handler: EventHandler<T>): void
}

abstract class EventHandler<T extends DomainEvent> {
  abstract readonly eventName: string
  abstract handle(event: T, ctx: EventHandlerContext):
    Promise<void>
  retries?: number = 3
  backoffMs?: number = 1000
}
```

18.4 Event Shape

```
abstract class DomainEvent {
  abstract readonly name: string
  readonly id: string // UUID v7
  readonly timestamp: Date
  readonly tenantId: string
  readonly correlationId: string // ties to request
    that triggered it
  readonly actorId: string | null
```

```
    abstract readonly payload: unknown
  }
```

18.5 Delivery Semantics

- **At-least-once delivery** — handlers must be idempotent
- **Outbox pattern** — events written to `event_outbox` table in same transaction as business mutation; published after commit. Guarantees no event lost if process crashes mid-transaction.
- **Async handlers** — handlers run on next tick to avoid blocking the request
- **Failure isolation** — if one handler throws, other handlers still run; failure logged + retried

18.6 Event Replay

`POST /api/_internal/events/replay` (admin only) — re-publishes events from outbox for a given time range. Used for: - Recovering from subscriber outages - Backfilling new subscribers with historical events - Debugging

19. Shared Types and Enums

19.1 Brand Types (Nominal Typing)

```
// packages/shared/src/types/brands.ts
declare const brand: unique symbol
type Brand<T, B extends string> = T & { readonly [brand]:
    B }

export type EntityId = Brand<string, 'EntityId'>
export type TenantId = Brand<string, 'TenantId'>
export type UserId = Brand<string, 'UserId'>
export type RoleId = Brand<string, 'RoleId'>
export type ProductId = Brand<string, 'ProductId'>
export type SupplierId = Brand<string, 'SupplierId'>
```

```
export type PurchaseOrderId = Brand<string,
    'PurchaseOrderId'>
export type GrnId = Brand<string, 'GrnId'>
export type BatchId = Brand<string, 'BatchId'>
// ... one per entity
```

Rationale: Prevents accidental cross-assignment (e.g., passing `UserId` where `TenantId` is expected). TypeScript enforces at compile time; zero runtime cost.

19.2 Common Types

```
interface Identifiable { id: string }
interface Tenanted { tenantId: string }
interface Auditable { createdAt: Date; createdBy: string |
    null; updatedAt: Date; updatedBy: string | null }
interface SoftDeletable { deletedAt: Date | null;
    deletedBy: string | null }
interface Versioned { version: number }
```

19.3 Enums

All enums live in `packages/shared/src/enums/`. Example:

```
export enum AuditAction {
    CREATE = 'CREATE',
    UPDATE = 'UPDATE',
    DELETE = 'DELETE',
    APPROVE = 'APPROVE',
    REJECT = 'REJECT',
    TRANSITION = 'TRANSITION',
    LOGIN = 'LOGIN',
    LOGOUT = 'LOGOUT',
    EXPORT = 'EXPORT',
    PRINT = 'PRINT',
}
```

```
export enum Permission {  
  // Org  
  ORG_READ = 'org:read',  
  ORG_CREATE = 'org:create',  
  ORG_UPDATE = 'org:update',  
  ORG_DELETE = 'org:delete',  
  // Auth  
  AUTH_MANAGE_USERS = 'auth:manage_users',  
  AUTH_MANAGE_ROLES = 'auth:manage_roles',  
  // Product  
  PRODUCT_READ = 'product:read',  
  PRODUCT_CREATE = 'product:create',  
  // ... one per resource:action  
}
```

```
export enum EntityType {  
  ORGANIZATION = 'Organization',  
  USER = 'User',  
  ROLE = 'Role',  
  PRODUCT = 'Product',  
  SUPPLIER = 'Supplier',  
  PURCHASE_ORDER = 'PurchaseOrder',  
  GRN = 'GoodsReceipt',  
  // ...  
}
```

```
export enum NotificationChannel {  
  IN_APP = 'IN_APP',  
  EMAIL = 'EMAIL',  
  SMS = 'SMS',  
  WHATSAPP = 'WHATSAPP',  
}
```

```
export enum WorkflowAction {  
  SUBMIT = 'SUBMIT',  
  APPROVE = 'APPROVE',  
  REJECT = 'REJECT',  
}
```

```
CANCEL = 'CANCEL',  
RECEIVE = 'RECEIVE',  
CLOSE = 'CLOSE',  
}
```

20. Shared DTOs

20.1 Pagination

```
interface PaginationParams {  
    page: number           // 1-indexed, default 1  
    pageSize: number       // default 25, max 200  
}  
  
interface PaginatedResult<T> {  
    data: T[]  
    meta: {  
        page: number  
        pageSize: number  
        total: number  
        totalPages: number  
        hasNext: boolean  
        hasPrev: boolean  
    }  
}
```

20.2 Filtering & Sorting

```
interface FilterOperator {  
    eq?: unknown  
    ne?: unknown  
    in?: unknown[]  
    notIn?: unknown[]  
    gt?: unknown
```

```

gte?: unknown
lt?: unknown
lte?: unknown
contains?: string
startsWith?: string
endsWith?: string
between?: [unknown, unknown]
isNull?: boolean
isNotNull?: boolean
}

type Filter<T> = {
  [K in keyof T]?: FilterOperator | Filter<T[K]>
} & {
  AND?: Filter<T>[]
  OR?: Filter<T>[]
  NOT?: Filter<T>
}

interface SortParam {
  field: string
  direction: 'asc' | 'desc'
}

interface ListQuery {
  pagination?: PaginationParams
  filter?: Filter<any>
  sort?: SortParam[]
  search?: string // full-text search query
  fields?:
    string[] // field selection (sparse
    fieldsets)
}

```


20.3 Audit Log DTO

```
interface AuditLogDto {
  id: string
  timestamp: string
  tenantId: string
  actorType: 'USER' | 'SYSTEM' | 'API_KEY' | 'JOB'
  actorId: string | null
  actorName: string | null
  actorRole: string | null
  action: AuditAction
  severity: 'INFO' | 'WARN' | 'CRITICAL'
  entityType: string
  entityId: string | null
  entityCode: string | null
  before: object | null
  after: object | null
  diff: object | null
  reason: string | null
  correlationId: string
}
```

20.4 Response Envelope DTOs (see §13.3)

21. Shared Domain Events

21.1 Event Catalog

All events catalogued in `packages/shared/src/events/event-names.ts`.
Examples:

```
export const EventName = {
  // Auth
  UserRegistered: 'UserRegistered',
```

```
UserLoggedIn: 'UserLoggedIn',  
UserLoggedOut: 'UserLoggedOut',  
UserLocked: 'UserLocked',  
RoleAssigned: 'RoleAssigned',  
RoleRevoked: 'RoleRevoked',
```

// Organization

```
CompanyCreated: 'CompanyCreated',  
PlantActivated: 'PlantActivated',  
PlantDeactivated: 'PlantDeactivated',
```

// Procurement

```
SupplierCreated: 'SupplierCreated',  
SupplierBlacklisted: 'SupplierBlacklisted',  
PurchaseOrderSubmitted: 'PurchaseOrderSubmitted',  
PurchaseOrderApproved: 'PurchaseOrderApproved',  
PurchaseOrderRejected: 'PurchaseOrderRejected',  
PurchaseOrderSent: 'PurchaseOrderSent',
```

// Goods Receipt

```
GrnCreated: 'GrnCreated',  
GrnPosted: 'GrnPosted',  
GrnQualityHold: 'GrnQualityHold',  
GrnPutawayComplete: 'GrnPutawayComplete',
```

// Quality

```
IqcInspectionStarted: 'IqcInspectionStarted',  
IqcInspectionPassed: 'IqcInspectionPassed',  
IqcInspectionFailed: 'IqcInspectionFailed',  
NcrCreated: 'NcrCreated',  
CapaCreated: 'CapaCreated',  
CapaVerified: 'CapaVerified',  
CoaGenerated: 'CoaGenerated',  
CoaSigned: 'CoaSigned',
```

// Stock

```
StockPosted: 'StockPosted',
```

```

    StockReversed: 'StockReversed',
    StockLow: 'StockLow',
    StockExpired: 'StockExpired',

    // System
    SystemError: 'SystemError',
    SlowTransaction: 'SlowTransaction',
    JobFailed: 'JobFailed',
  } as const

  export type EventName = typeof EventName[keyof typeof
    EventName]

```

21.2 Event Payload Contracts

Each event has a typed payload defined in `packages/shared/src/events/payloads/`. Example:

```

// payloads/purchase-order.payloads.ts
export interface PurchaseOrderSubmittedPayload {
  poId: string
  poNumber: string
  supplierId: string
  supplierName: string
  totalValue: number
  currency: string
  submittedBy: string
  approverRoleIds: string[]
}

export interface PurchaseOrderApprovedPayload {
  poId: string
  poNumber: string
  approvedBy: string
  approvedAt: string
}

```

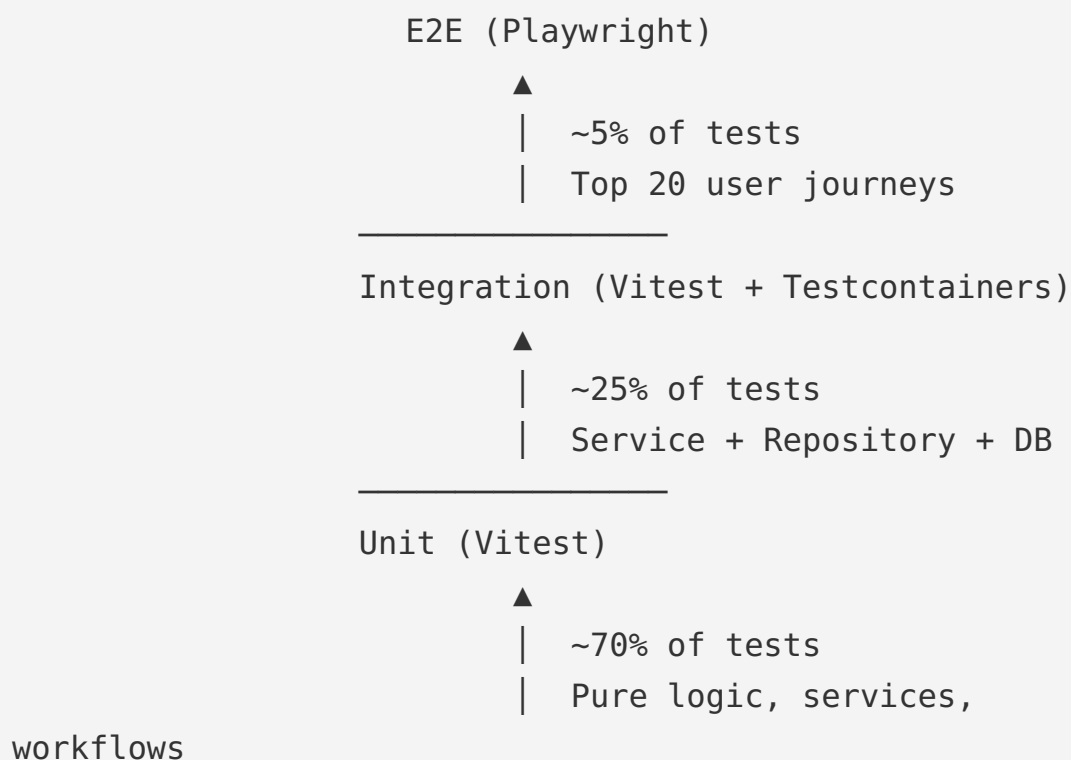
```
nextAction: 'SEND_TO_SUPPLIER' | 'INTERNAL_REVIEW'  
}
```

21.3 Event Versioning

Every event payload includes `version: 1` field. Breaking changes require bumping version + adding new payload type. Subscribers handle both versions during migration window.

22. Testing Strategy

22.1 Test Pyramid



22.2 Unit Tests

- **Target:** 70% coverage for business code, 90% for foundation code
- **Speed:** <10s for full unit suite
- **Tools:** Vitest

- **What to test:**

- Service business rules (validation, calculations, state machines)
- Workflow transitions (valid + invalid)
- Validation schemas (zod)
- Pure utility functions
- Error handling paths

- **What NOT to test:**

- Prisma queries (tested in integration)
- HTTP layer (tested in integration)
- UI rendering (tested in component tests)

22.3 Integration Tests

- **Target:** Every endpoint has at least one happy-path + one error-path integration test
- **Tools:** Vitest + Testcontainers (real PostgreSQL in Docker)
- **Setup:** Per-test transaction rollback (no test pollution)
- **What to test:**
 - Full request → response cycle
 - Repository queries against real DB
 - Multi-tenant isolation (user from tenant A cannot read tenant B's data)
 - Workflow transitions persist correctly
 - Audit log entries written
 - Domain events published

22.4 End-to-End Tests

- **Target:** Top 20 user journeys
- **Tools:** Playwright
- **Examples:**
 - Login → create supplier → create PO → approve PO → receive GRN → IQC pass → stock available
 - Login → create NCR → auto-quarantine batch → create CAPA → verify effectiveness → close
 - Login → generate COA → sign → distribute → archive

22.5 Performance / Load Tests

- **Tools:** k6
- **Targets** (per audit acceptance criteria):
 - Recall identification: <5s for 10M inventory records
 - COA generation: <3s end-to-end
 - Dashboard load: <2s
 - List endpoints: p95 <500ms
- **Run:** Nightly in CI; alert on regression >20%

22.6 Mutation Tests

- **Tools:** Stryker
- **Target:** Foundation code only (workflow engine, audit, permissions)
- **Purpose:** Verify tests actually catch bugs (not just line coverage)

22.7 Contract Tests

- Frontend and backend share `@suop/shared` types
- CI runs `tsc --noEmit` on both — type mismatch fails the build
- OpenAPI spec generated from zod schemas (via `zod-to-openapi`)
- Frontend codegen from OpenAPI spec as a CI check (no drift)

22.8 Test Database

- Dedicated test database, reset between test runs
- Seed data versioned in `tests/fixtures/`
- No mocking of Prisma — use real DB via Testcontainers
- Mock only external services (S3 mock via MinIO, email mock via MailHog)

22.9 Coverage Gate

- **Foundation code (`src/core/`):** ≥90% line coverage, ≥80% branch
- **Business code (`src/modules/`):** ≥70% line coverage, ≥60% branch
- CI fails PRs below threshold
- Coverage trend tracked over time; alert on >5% drop

22.10 Test Naming Convention

```
describe('PurchaseOrderService.create', () => {
  describe('when supplier is active', () => {
    it('creates the purchase order in DRAFT status')
    it('assigns the requester as createdBy')
    it('publishes PurchaseOrderCreated event')
  })
  describe('when supplier is inactive', () => {
    it('throws BusinessRuleError with code
        PO.SUPPLIER_INACTIVE')
  })
  describe('when total value exceeds budget', () => {
    it('throws BusinessRuleError with code
        PO.BUDGET_EXCEEDED')
  })
})
```

23. Coding Standards

23.1 TypeScript Configuration

- `strict: true`
- `noUncheckedIndexedAccess: true`
- `exactOptionalPropertyTypes: true`
- `noFallthroughCasesInSwitch: true`
- `noImplicitReturns: true`
- Target: ES2022
- Module: NodeNext (backend), Bundler (frontend)

23.2 Naming Conventions

Element	Convention	Example
	PascalCase	<code>OrganizationTree.tsx</code>

Element	Convention	Example
Files (components)		
Files (utilities, hooks)	kebab-case	<code>use-organizations.ts</code>
Files (classes, types)	PascalCase	<code>BaseRepository.ts</code>
Variables, functions	camelCase	<code>purchaseOrderService</code>
Constants	SCREAMING_SNAKE_CASE	<code>MAX_PAGE_SIZE</code>
Types, Interfaces, Classes	PascalCase	<code>PurchaseOrderService</code>
Enums	PascalCase (type), SCREAMING_SNAKE_CASE (values)	<code>enum AuditAction { CREATE = 'CREATE' }</code>
Database tables	snake_case	<code>purchase_orders</code>
Database columns	snake_case	<code>created_at</code>
API paths	kebab-case	<code>/api/purchase- orders/:id</code>
Environment variables	SCREAMING_SNAKE_CASE	<code>DATABASE_URL</code>

23.3 File Conventions

- One default export per file for components; named exports for utilities
- Max file length: 400 lines (split if larger)
- Max function length: 50 lines (extract if larger)
- No `any` type without `// eslint-disable-next-line` with justification

- No `// @ts-ignore` — fix the type or use `// @ts-expect-error` with reason

23.4 Import Order (enforced by ESLint)

```
// 1. Node built-ins
import { readFile } from 'node:fs/promises'

// 2. External packages
import { z } from 'zod'
import express from 'express'

// 3. Internal packages (@suop/*)
import { Permission, EntityType } from '@suop/shared'
import { prisma } from '@suop/prisma'

// 4. Internal absolute (@/...)
import { BaseService } from '@core/services'
import { logger } from '@core/logging'

// 5. Relative (./ or ../)
import { PoStatus } from './types'
import { CreatePoSchema } from './schemas'
```

23.5 Layer Boundary Rules (enforced by ESLint)

```
// .eslintrc.js – no-restricted-imports per layer
 '@suop/eslint-config/no-cross-layer-imports': 'error'

// Rules:
// - controllers/ cannot import from repositories/
// - repositories/ cannot import from services/
// - services/ cannot import from controllers/
// - modules/a/ cannot import from modules/b/ (use event bus
  instead)
```

23.6 Commit Message Convention (Conventional Commits)

```
<type>(<scope>): <subject>
```

```
<body>
```

```
<footer>
```

Types: `feat`, `fix`, `refactor`, `test`, `docs`, `chore`, `perf`, `ci` Scope: module name (e.g., `purchase-order`, `auth`, `core-workflow`)

Examples:

```
feat(purchase-order): implement P0 submission workflow
fix(stock-ledger): correct FEFO batch selection edge case
refactor(core-audit): extract audit logger from middleware
test(iqc): add integration tests for fail disposition
docs(architecture): add Phase 0 foundation document
```

23.7 Pull Request Standards

- PR title matches commit convention
- PR description includes: What, Why, How, Testing Done, Screenshots (if UI)
- Max 500 lines changed per PR (split larger work)
- At least one reviewer approval required
- All CI checks green
- No `console.log` in merged code

23.8 Branching Strategy

- `main` — always deployable to production
- `develop` — integration branch
- `feature/<scope>-<description>` — feature branches
- `fix/<scope>-<description>` — bug fix branches

- `hotfix/<scope>-<description>` — production hotfixes (branched from `main`, merged to both `main` and `develop`)

23.9 Code Review Checklist

- ☐ No `any` types
 - ☐ No hardcoded values (use config or constant)
 - ☐ All mutations audit-logged
 - ☐ All endpoints have RBAC
 - ☐ All status changes go through workflow engine
 - ☐ Tests added for new logic
 - ☐ No secrets in code
 - ☐ Error messages user-friendly (no stack traces leaked)
 - ☐ Multi-tenant scoping verified
 - ☐ Soft delete used (no hard delete)
-

24. Definition of Done — Phase 0

Phase 0 is complete when **all** of the following are true:

24.1 Code Implemented (but no business modules)

- ☐ Folder structure created per §3
- ☐ `@suop/shared` package with all types, enums, DTOs, events
- ☐

`@suop/prisma` package with client + extensions (soft delete, tenant scope, audit log)

☐

`@suop/ui` package with base components

☐

Backend foundation:

☐

Config framework (env validation, feature flags)

☐

Error framework (full hierarchy + error codes)

☐

Response envelope

☐

Logging framework (Pino + request context)

☐

Validation framework (zod + registry)

☐

Audit framework (logger + middleware + immutable table)

☐

Event bus (with outbox pattern)

☐

Notification engine (in-app + email channels; SMS/WhatsApp stubbed)

☐

Workflow engine (state machine + registry)

☐

Permission framework (middleware + decorator + registry)

☐

Tenant middleware

☐

Transaction helper

☐

File upload service (S3 driver + local driver)

☐

Background job framework (BullMQ + scheduler)

☐

Base repository, base service, base controller classes

☐

Frontend foundation:

☐

- ☐ Axios client with interceptors
- ☐ React Query client + query key factory
- ☐ Auth context + permission guard
- ☐ Shared UI components (DataTable, EntityForm, StatusBadge, etc.)

24.2 Tests

- ☐ Foundation code unit test coverage $\geq 90\%$
- ☐ Integration tests for every middleware
- ☐ E2E test for login flow (only real flow available)
- ☐ Test database setup via Testcontainers
- ☐ CI pipeline running all tests on every PR

24.3 Documentation

- ☐ This document approved and committed to `docs/architecture/`
- ☐ README.md in every package (`@suop/shared` , `@suop/prisma` , etc.)
- ☐ Architecture Decision Records (ADRs) for major decisions
- ☐ Runbook for common ops tasks (restart, restore backup, etc.)

24.4 DevOps

- ☐ `Dockerfile.web` and `Dockerfile.backend` (multi-stage builds)
- ☐ `docker-compose.yml` for local dev (Postgres + Redis + MinIO + MailHog + backend + web)
- ☐

CI pipeline: lint → typecheck → test → build



CD pipeline: staging on `develop` merge, production on `main` merge
(manual approval)



Sentry integration (DSN configured, source maps uploaded)

24.5 Acceptance Tests (Proof Phase 0 Works)

Before any business module work begins, the following must be demonstrable end-to-end:

1. `docker-compose up` starts the entire stack
2. Visit `http://localhost:3000` — landing page loads
3. Visit `http://localhost:3030/api/health` — returns `{ status: 'ok' }`
4. Visit `http://localhost:3030/api/_internal/schemas` — returns list of registered schemas
5. Visit `http://localhost:3030/api/_internal/jobs` — returns empty job list (requires auth → 401 without token)
6. Run `npm test` — all foundation tests pass
7. Run `npm run test:e2e` — login e2e test passes
8. Run `npm run build` — both apps build without errors
9. Audit log table exists in DB with row-level security policy
10. Event outbox table exists
11. Notification outbox table exists
12. Soft-delete Prisma extension verified (delete on a test record sets `deletedAt`, doesn't actually delete)

25. Open Questions for Approval

The following decisions are required before Phase 0 implementation begins. Please approve or modify each:

Q1 — Monorepo Tooling

Recommendation: Turborepo + pnpm workspaces. **Why:** Industry standard for TypeScript monorepos; intelligent caching; simple config. **Alternative:** Nx (more features, more complexity). **Approve / modify?**

Q2 — Backend HTTP Framework

Recommendation: Hono (lightweight, fast, runs on Bun). **Why:** Bun-native, fast, simple middleware composition. **Alternative:** Express (more familiar, more middleware available). **Approve / modify?**

Q3 — Runtime

Recommendation: Bun (already in use). **Why:** Faster startup, built-in test runner, native TypeScript. **Alternative:** Node.js 20 LTS (more mature ecosystem). **Approve / modify?**

Q4 — Database Hosting

Recommendation: Supabase (managed Postgres + free tier sufficient for dev). **Why:** Free managed Postgres with backups; integrates with auth if needed later. **Alternative:** Self-hosted Postgres in Docker (more control, more ops burden). **Decision needed:** Provide Supabase project URL + service key, or instruct self-hosted?

Q5 — Redis Hosting

Recommendation: Upstash Redis (serverless, free tier) for production; in-memory for dev/tests. **Why:** Required for rate limiting, job queue (BullMQ), distributed locks. **Approve / modify?**

Q6 — File Storage

Recommendation: MinIO (S3-compatible, self-hosted in docker-compose) for dev; AWS S3 or Supabase Storage for production. **Why:** S3 API is the de facto standard; MinIO lets dev match prod exactly. **Approve / modify?**

Q7 — Email Provider

Recommendation: MailHog (dev) → AWS SES (prod). **Why:** SES is cheapest at scale; MailHog captures emails locally for inspection.

Alternative: SendGrid (more features, more cost), Postmark. **Approve / modify?**

Q8 — Test Database Strategy

Recommendation: Testcontainers — spin up real Postgres in Docker per test run. **Why:** Tests against real DB; no SQLite/Postgres drift; isolated.

Alternative: Shared test database with transaction rollback (faster, riskier). **Approve / modify?**

Q9 — Frontend State Management

Recommendation: React Query (server state) + Zustand (client UI state).

Why: React Query is the standard for server state; Zustand is minimal for UI-only state (theme, sidebar, etc.). **Alternative:** Redux Toolkit (heavier, more opinionated). **Approve / modify?**

Q10 — Existing Code Disposition

The current `src/app/page.tsx` (37,080 lines, 236 modules, all hardcoded) and `mini-services/suop-backend/index.ts` (14,142 lines, all mock) will not be migrated as-is. Recommendation:

- **Keep** Prisma schema as starting point (will be refined per Schema Review Report)
- **Keep** UI component library (shadcn/ui base)
- **Discard** all mock data arrays (replaced by real API calls)
- **Discard** all backend mock endpoints (replaced by real implementations)
- **Keep** login screen UI (but wire to real auth)
- **Keep** sidebar shell (but rebuild module routing)

Approve / modify?

Q11 — Demo Mode

Recommendation: Remove demo mode entirely from production builds. Demo mode only available when `NODE_ENV=development` and `ENABLE_DEMO_MODE=true`. **Why:** Demo mode is a security risk in production (anyone can click “Explore Demo” and bypass auth). **Approve / modify?**

Q12 — API Versioning

Recommendation: Prefix all routes with `/api/v1/...`. Breaking changes go to `/api/v2/...` with the old version maintained for 6 months. **Why:** Mobile app and any external integrations need stable contracts. **Approve / modify?**

Q13 — Logging Vendor

Recommendation: Pino (structured JSON logs) → stdout → collected by Docker → shipped to Loki/Datadog. **Why:** Pino is fastest; structured logs are greppable; no vendor lock-in. **Alternative:** Winston (slower, more plugins). **Approve / modify?**

Q14 — Monitoring Vendor

Recommendation: Sentry for error tracking; Prometheus + Grafana for metrics; Loki for logs. **Why:** Sentry has best-in-class TS support; Prometheus/Grafana/Loki are open-source and self-hostable. **Alternative:** Datadog (all-in-one, paid). **Approve / modify?**

Q15 — Implementation Cadence

Recommendation: After Phase 0 is approved, I implement Phase 0 code and report back with the standard output format (Database/Backend/Frontend/Workflow/Testing progress %). You review, approve, then I begin Phase 1 (Organization module).

Alternative: Time-boxed sprints (2 weeks each) with demo at end of each. **Approve / modify?**

Approval Block

By approving this document, the Project Owner authorizes:

1. The architectural principles, folder structure, and layering rules defined herein
2. The shared foundation framework specifications (§§6-18)
3. The shared types, enums, DTOs, and events (§§19-21)
4. The testing strategy (§22)
5. The coding standards (§23)
6. The Definition of Done for Phase 0 (§24)
7. The implementation order: Phase 0 (foundation) → Phase 1 (Organization) → ... per the roadmap

Approval does **not** authorize: - Database schema changes (separate Schema Review Report required) - Stock Ledger implementation (separate Stock Ledger Design document required) - Any business module code

Approved by: _____ **Date:** _____

Signature: _____

End of Phase 0 Architecture Document